

Hybrid Pattern Recognition System

About the Project	1
Technology Stack	2
Datasets	2
Algorithms & Models	2
How the algorithms work	3
Problem & Market Opportunity	4
Target Customers	5
Competitive Advantage	5

About the Project

PatternScope AI is the core technology foundation (first step) for an academic handwriting authentication and automated grading system. The end goal is a platform where professors upload handwritten exams and assignments, and the system validates that the student who signed the submission actually wrote it by matching against their baseline handwriting patterns, then grades the content by extracting text from handwritten answers, analyzing topic coverage, and computing similarity scores against professor-provided answer keys.

The current implementation builds this foundation by recognizing handwritten digits, geometric shapes, and custom symbols. Users upload images, the system returns predictions with confidence scores, compares multiple AI models, and learns from user feedback in real-time. In the future, we plan to extend this project to handle full text recognition, writer verification, and content-based grading.

Technology Stack

The system is built with C++14, CMake, stb_image, and natID GUI framework.

Datasets

The system trains on three datasets. The mnist folder contains MNIST handwritten digit data with 60,000 training images and 10,000 test images in IDX format. The datasets folder contains Quick Draw data in numpy format with 1,000 training examples and 200 test examples per category. Shapes include circle, square, and triangle. Symbols include checkmark, cross, and heart. The system uses a train-test split (see trainer.cpp). The training process uses these datasets to build pattern libraries for KNN, calculate statistics for Naive Bayes, learn weights for MLP, and select templates for A*.

Algorithms & Models

The system uses five algorithms working together. K-Nearest Neighbors provides baseline classification. Naive Bayes handles probabilistic predictions. Mini Multi-Layer Perceptron captures nonlinear patterns. A* Template Matcher offers decision explanations. Rule-Based Engine acts as fallback when confidence is low.

1. K-Nearest Neighbors (KNN)
2. Naive Bayes
3. Mini Multi-Layer Perceptron (MLP)
4. A* Template Matcher
5. Rule-Based Engine (Fallback)

How the algorithms work

1) K-NEAREST NEIGHBORS (KNN)

KNN algorithm stores all training examples as feature vectors. When classifying a new pattern, it finds the 3 closest training examples by Euclidean distance. The algorithm returns the most common label among those 3 neighbors. Confidence is based on neighbor agreement and distance. In this project, KNN serves as a baseline classifier. It benefits most from active learning since new examples immediately expand the training library, see (src/ai/models/KNN.cpp).

2) NAIVE BAYES

Naive Bayes algorithm calculates mean and variance of pixel intensities for each class during training. At prediction time, it computes the probability that the input belongs to each class using Gaussian distribution. The algorithm returns the class with the highest probability. In this project, Naive Bayes is a fast probabilistic classifier. It handles grayscale patterns by modeling pixel distributions independently.

3) MINI MULTI-LAYER PERCEPTRON

This neural network has 784 input neurons for 28x28 pixels, 64 hidden neurons, and output neurons that vary by task. The output layer has 10 neurons for digit recognition because there are 10 possible classes (digits 0 through 9). For shapes and symbols, the output layer has 3 neurons because each task has 3 distinct classes, see Engine.cpp. Each output neuron represents one class, and the neuron with highest activation determines the prediction. The network learns through backpropagation by adjusting weights to minimize classification error over training epochs. In this project, MLP captures nonlinear relationships between pixels. It provides higher accuracy on complex patterns where linear models fail.

4) A* TEMPLATE MATCHER

This algorithm uses the first 200 examples from training data as templates, limiting the set for efficiency. When classifying a new pattern, the algorithm calculates the Euclidean distance between the input and every template in memory. It uses a priority queue to sort templates by distance and returns the label of the closest match. In this project, A* provides explainable and deterministic decisions. Unlike neural networks that are black boxes, A* can identify exactly which training example the input matched to. Users can visualize the matched template to understand why the prediction was made (see src/ai/search/AStarMatcher.cpp).

5) RULE-BASED ENGINE

This engine analyzes structural features including edge count, corner count, aspect ratio, and symmetry. It applies if-then rules such as "4 corners plus rectangular shape equals 0 or 8". In this project, the rule engine is a fallback mechanism when all ML models have low confidence (below 40%). This ensures the system never fails silently (see src/ai/rules/RuleEngine.cpp).

6) ENSEMBLE & ACTIVE LEARNING

All four ML models vote on each prediction. Each model returns both a predicted label and a confidence score. The system accumulates confidence scores for each label. If KNN predicts "7" with 0.8 confidence and MLP predicts "7" with 0.6 confidence, label "7" receives a total score of 1.4. The label with the highest accumulated score wins. Final confidence is calculated by dividing the winning score by 4.0 since four models contribute. If this averaged confidence falls below 0.4, the rule engine takes over as fallback.

When users provide feedback on predictions, the system updates models in memory without retraining from scratch. KNN appends the new feature vector and label to its internal training data array. Naive Bayes recalculates class means and variances incrementally using the new example. MLP performs one forward and backward pass with the corrected example, adjusting weights immediately. These changes persist only in RAM during the session.

7) MODEL TRAINING

Training happens offline before the application runs (see readme). The system uses three datasets. The mnist folder contains MNIST handwritten digit data with 60,000 training images and 10,000 test images in IDX format. The datasets folder contains Quick Draw data in numpy format with hand-drawn shapes (circle, square, triangle) and symbols (checkmark, cross, heart). Each dataset has corresponding test data for validation.

The training process differs by model. KNN stores all training examples as feature vectors with no parameter learning. Naive Bayes calculates mean and variance for each pixel position across all examples in each class. These statistics become the trained parameters. MLP adjusts weights and biases through backpropagation. The network initializes weights randomly between -0.5 and 0.5. Training runs for 5 epochs with a learning rate of 0.01. Each epoch processes the entire dataset, computing forward pass activations and backward pass weight updates. The MLP learns two weight matrices. The first connects 784 input neurons to 64 hidden neurons. The second connects 64 hidden neurons to output neurons. Each layer also has bias parameters that shift neuron activations.

After training, all models serialize their parameters to text files in the models directory. KNN saves feature vectors and labels. Naive Bayes saves means and variances. MLP saves weight matrices and bias vectors. A* saves selected template vectors. The application loads these files at runtime, eliminating the need to retrain during user sessions.

Problem & Market Opportunity

Academic institutions process millions of handwritten assignments, exams, and forms annually, which is extremely time-consuming. Educational institutions need offline handwriting recognition that can be customized.

This system - when fully implemented - will address two academic needs. First, it authenticates student work by matching handwriting patterns to verify the registered student wrote the exam

or assignment. Second, it automates grading by comparing student responses against professor answer keys. Professors accept those that they consider well graded (reinforcement learning), and review only submissions where the system reports low confidence scores.

This project establishes the core technology foundation for the above-mentioned full solution. The current implementation recognizes digits, geometric shapes, and custom symbols. This pattern recognition engine will be extended to handle full handwriting analysis, character-by-character matching for authentication, and text comparison for grading automation.

Target Customers

Universities and colleges processing handwritten exams, assignments, and homework submissions. K-12 schools grading written worksheets and tests. Research institutions analyzing handwriting patterns in academic submissions.

Competitive Advantage

There is no similar software on the market. The system runs offline and eliminates per-page costs and data privacy risks. The multi-model architecture allows institutions to inspect how predictions are made and which models agree or disagree. The modular design allows swap of algorithms or add models. Active learning adapts to institution-specific handwriting patterns.